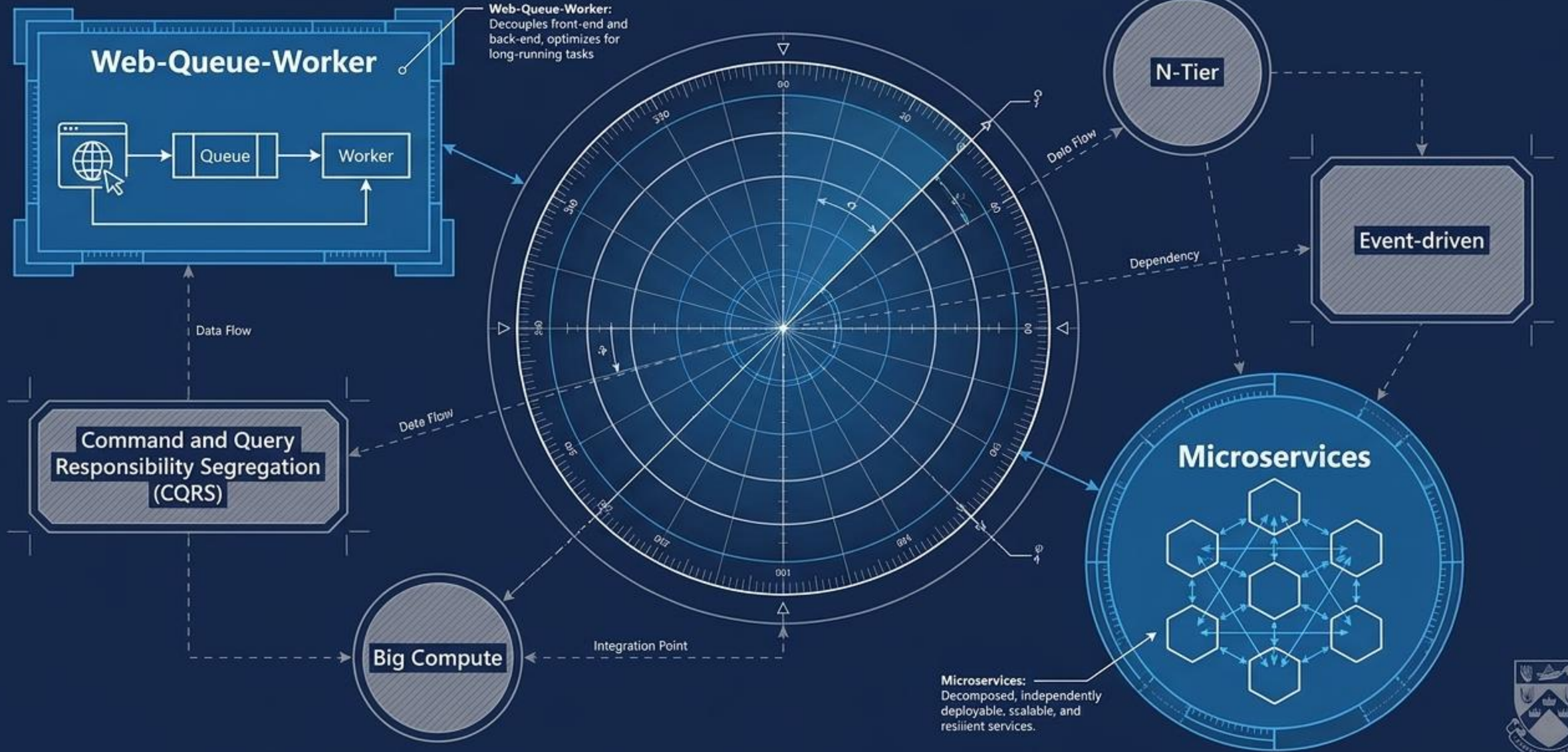
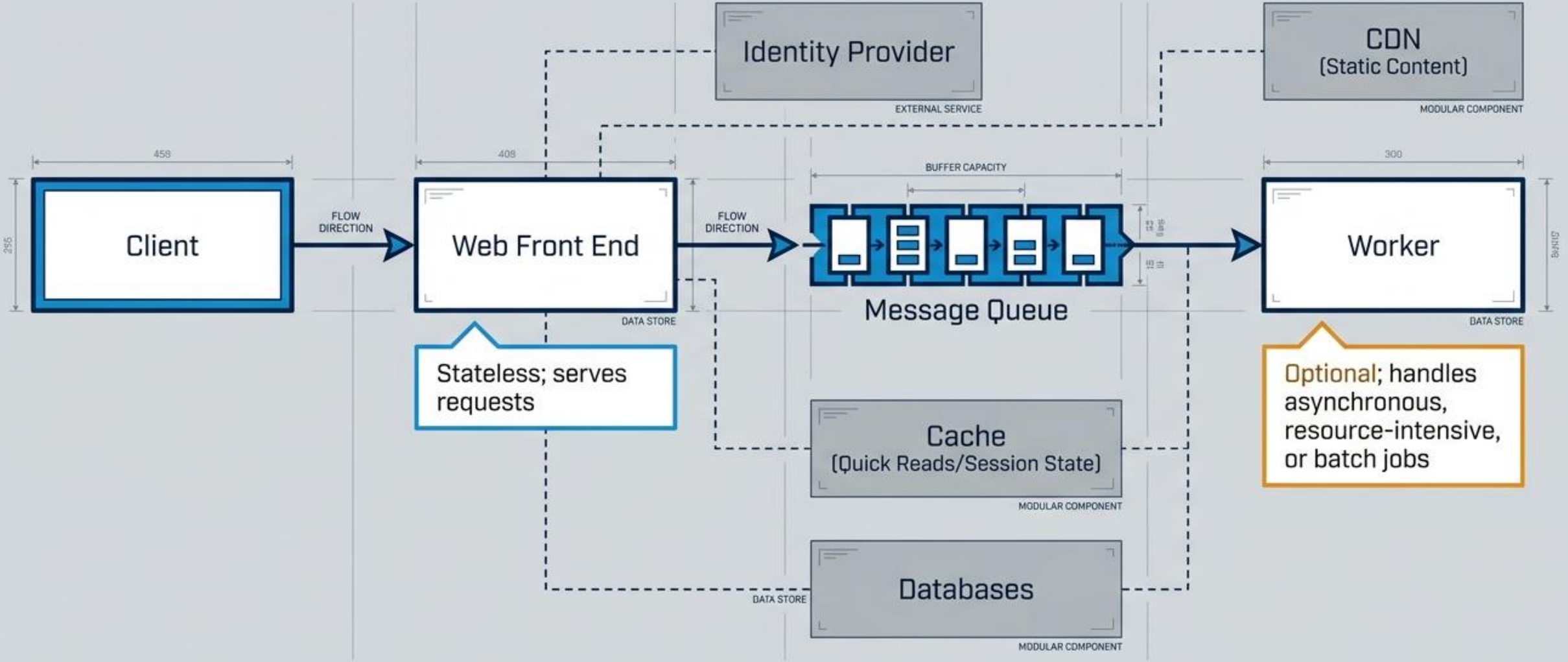
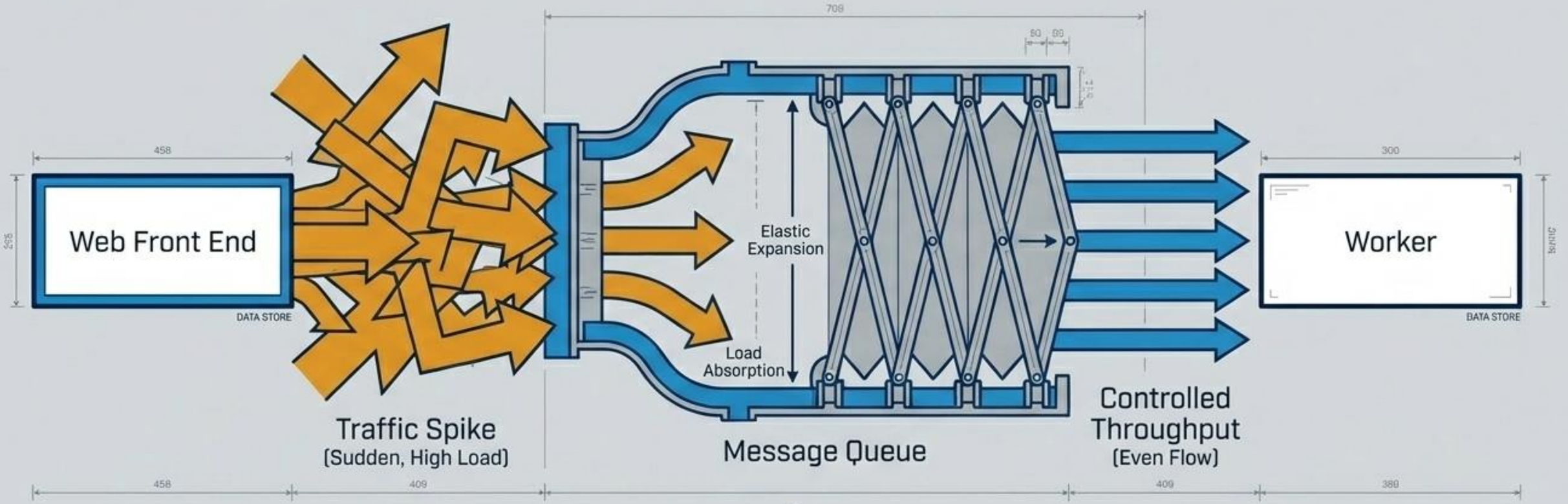
Mapping the Cloud Architecture Landscape



Web-Queue-Worker Provides Linear Decoupling for Moderate Complexity

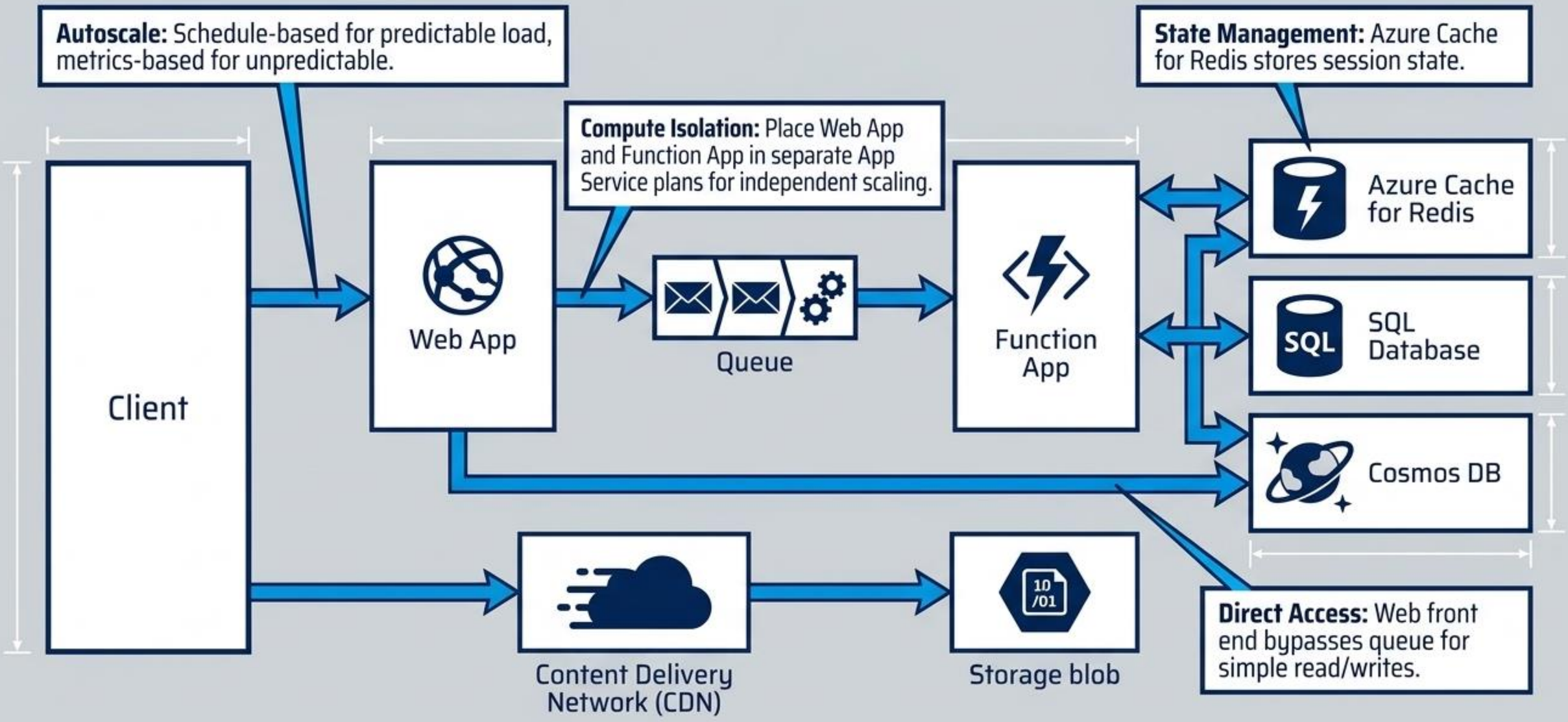


The Message Queue Acts as an Elastic Buffer Against Traffic Spikes



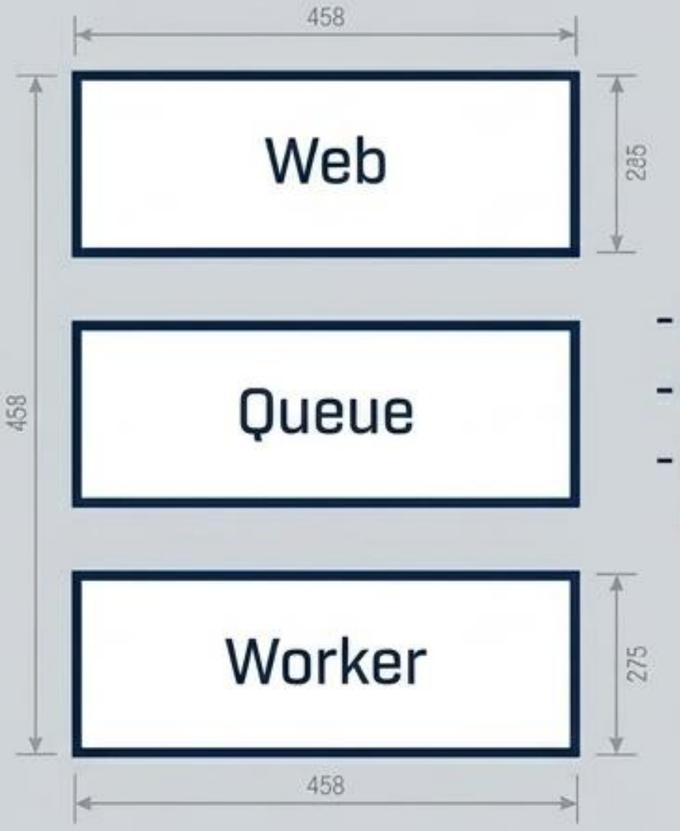
Without this async buffer, long-running operations would lock up the web front end, causing timeouts and system crashes.

Implementing Web-Queue-Worker via Azure Managed Compute



The Hidden Monolithic Risks of Web-Queue-Worker

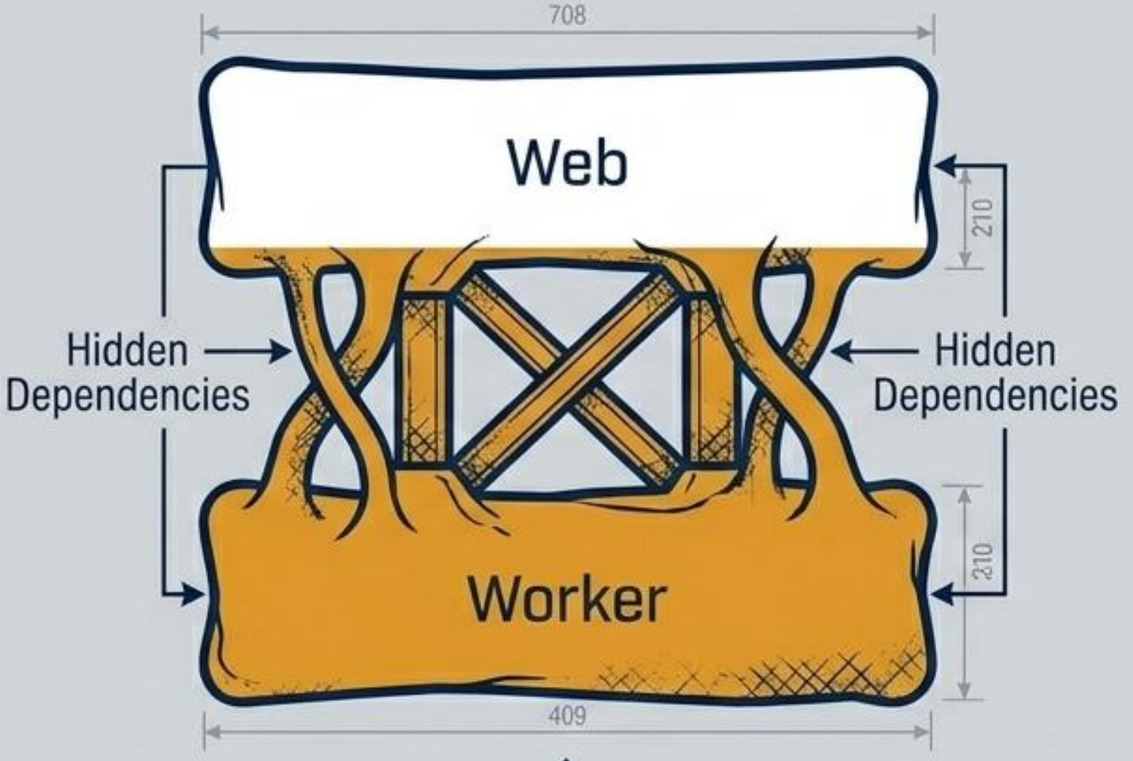
Clear Separation of Concerns



- Easy to deploy
- Simple to understand
- Independent scaling of front/back ends

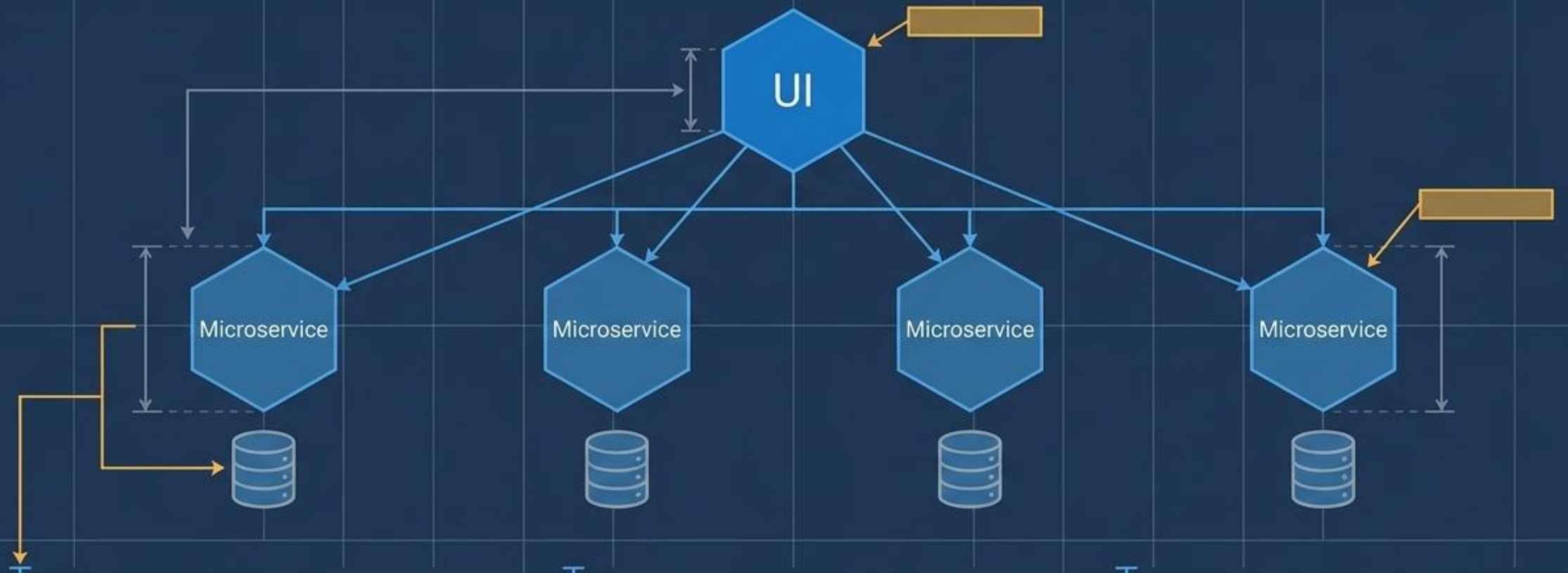
Editorial Blueprint

The Monolithic Creep



Without careful design, shared data schemas and shared code modules cause both the front end and worker to swell into difficult-to-maintain monolithic components.

Microservices Decentralize the Application Ecosystem



Independence

Small code bases. Deployed and updated without rebuilding the entire application.

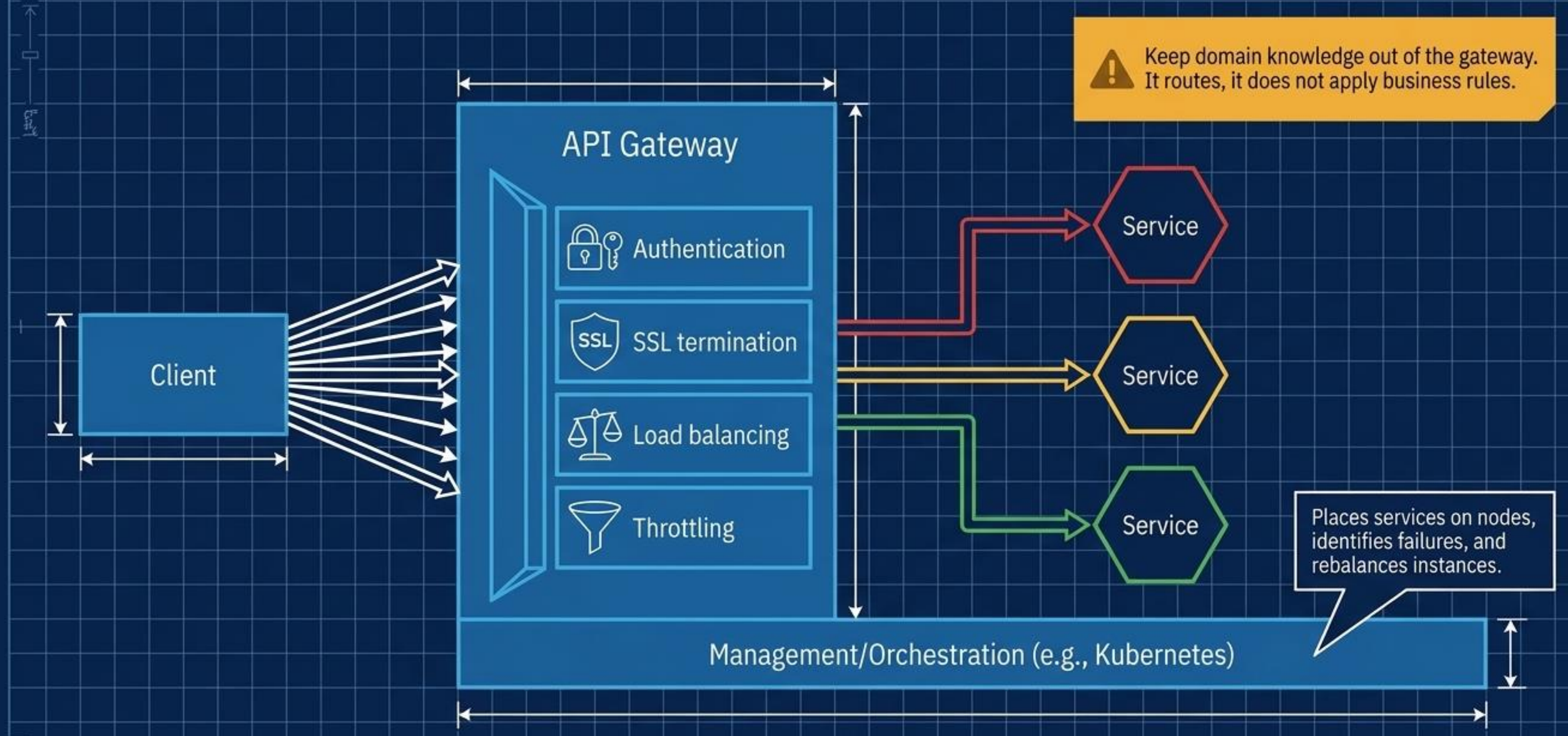
Autonomous Teams

Each service owned by a single, small, focused feature team.

Polyglot Capabilities

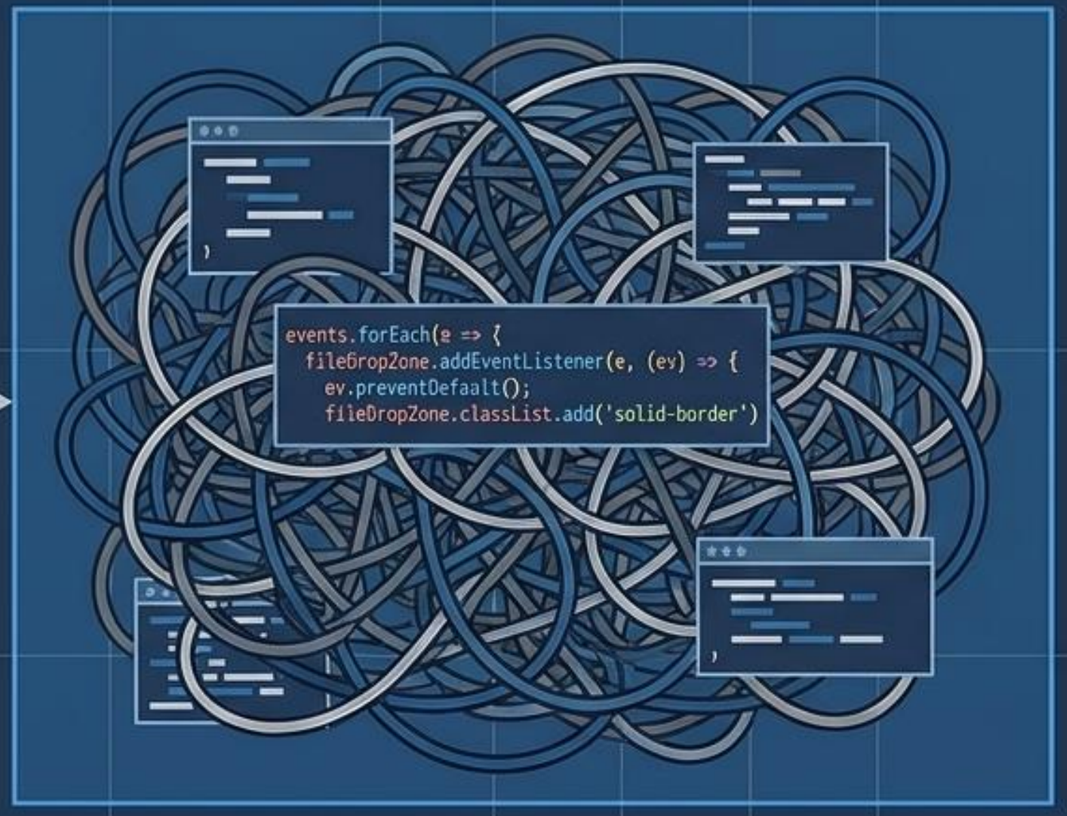
No forced tech stacks. Teams pick the language and framework that best fits the service.

Taming the Network with Orchestration and API Gateways



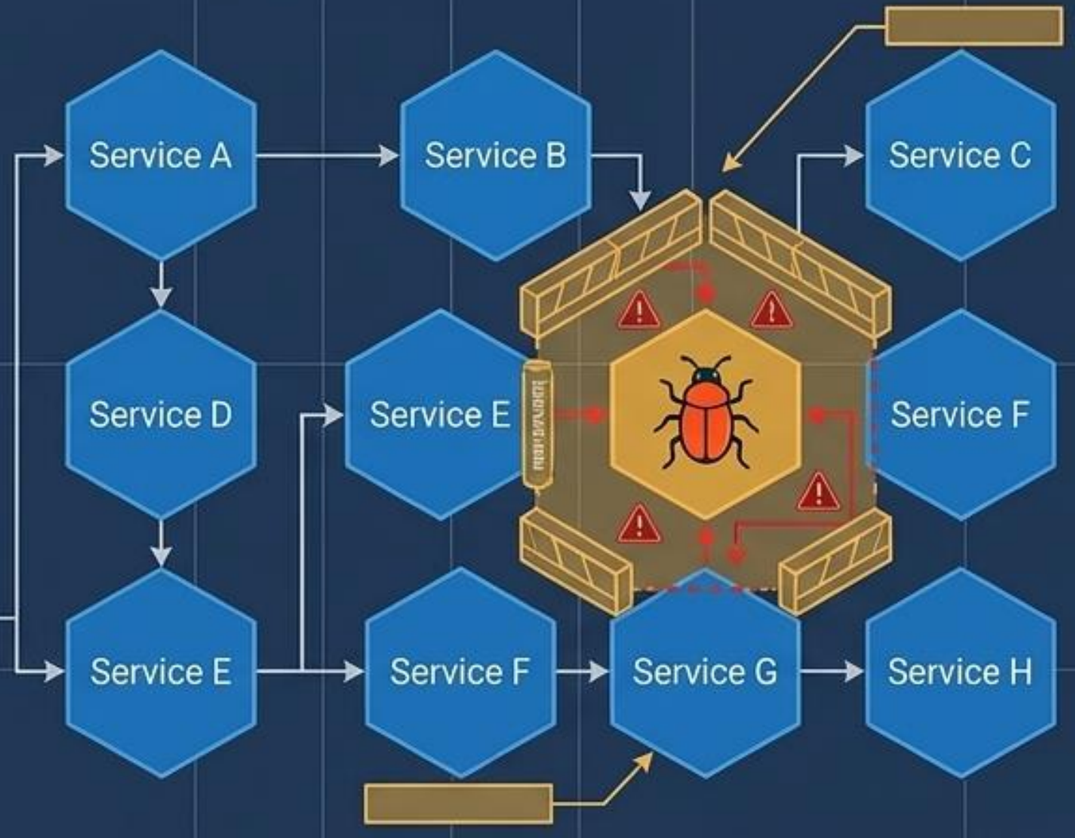
Containing Complexity and Isolating Faults

The Monolith Codebase



Adding a feature or fixing a bug here requires touching interconnected code everywhere, risking application-wide failure.

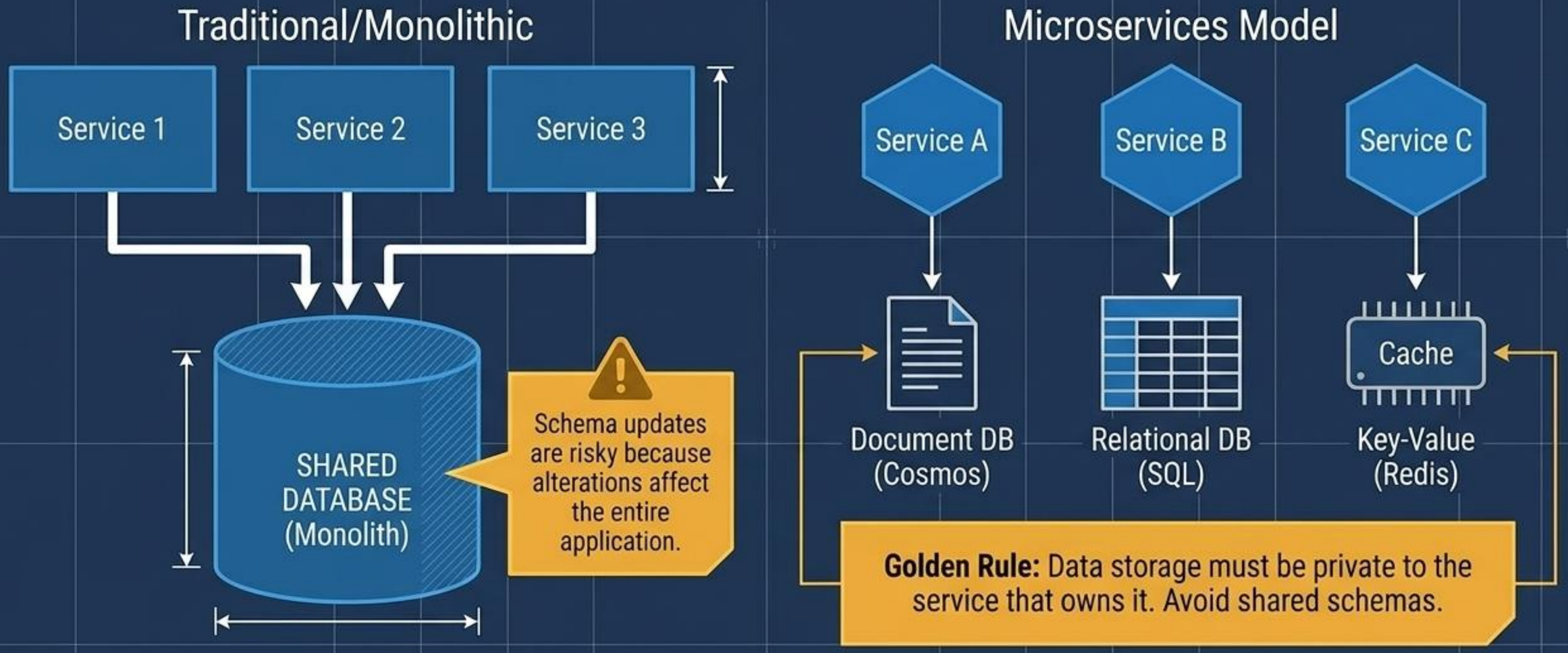
Microservices Isolation



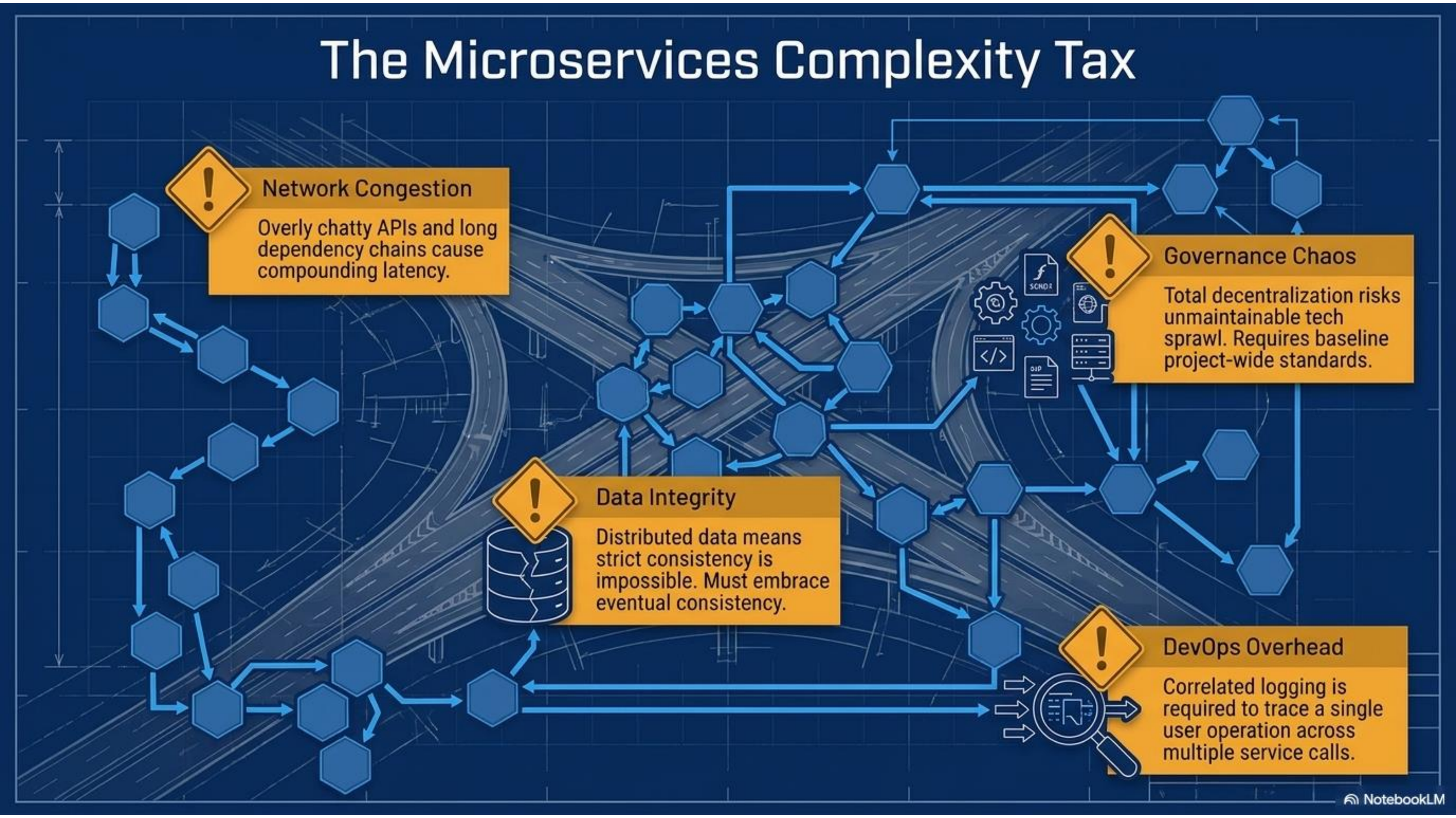
Fault is isolated to a single node. Small teams can push a fix without a full system redeploy.

Polyglot Persistence Ends the Shared Database Dependency

Data Funnel vs. Data Splinter



The Microservices Complexity Tax



The Architectural Diagnostic Matrix

Dimension	Web-Queue-Worker	Microservices
Domain Complexity	Simple to Moderate	Highly Complex / Evolving
Team Structure	Single, cohesive dev team	Multiple autonomous, small feature teams
Deployment	Coupled front/back updates	Completely independent deployments
Data Strategy	Shared databases common	Strictly isolated, polyglot persistence
Operational Overhead	Low, heavily managed via PaaS	High, requires mature DevOps and Orchestration

Core Directives for Modern Cloud Design

Model the Domain

Design APIs that reflect business domains, not internal technical implementation.

Decentralize Where Possible

Empower small teams. Avoid rigid communication protocols and shared data schemas that force tight coupling.

Design for Resilience

Isolate failures. Use asynchronous patterns like queue-based load leveling and circuit breakers to prevent cascading outages.

Offload Cross-Cutting Concerns

Keep business logic pure. Push authentication, SSL termination, and static caching to gateways and CDNs.

Diagnostic Framework derived from Cloud Design Patterns and Architecture. Contact: j.rafferty@ulster.ac.uk